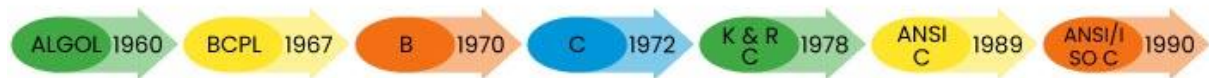


C Programming

C is a general-purpose procedural programming language initially developed by Dennis Ritchie in 1972 at Bell Laboratories of AT&T Labs. It was mainly created as a system programming language to write the UNIX operating system.

The history of the C programming language is quite fascinating and pivotal in the development of computer science and software engineering.

Year wise development of programming is as follows –



Why C Matters

- It is one of the most popular programming languages in the world.
- If you know C, you will have no problem learning other popular programming languages such as Java, Python, C++, C#, etc., as the syntax is similar.
- If you know C, you will understand how computer memory works.
- C is very fast, compared to other programming languages, like Java and Python.
- C is very versatile; it can be used in both applications and technologies.

Main features of C Language

- ❖ General Purpose and portable
- ❖ Procedural Programming Language
- ❖ Low-level Memory Access
- ❖ Fast Speed
- ❖ Clean Syntax

Application of C Language

- Gaming Engines
- Real Time Systems
- Embedded Systems
- Networking
- DBMS
- Medical devices
- Operating System

IDE for writing C code

An IDE (*Integrated Development Environment*) is used to edit and compile the code. Popular IDE's include **Code::Blocks**, **Eclipse**, and **Visual Studio**. These are all free, and they can be used to both edit and debug C code.

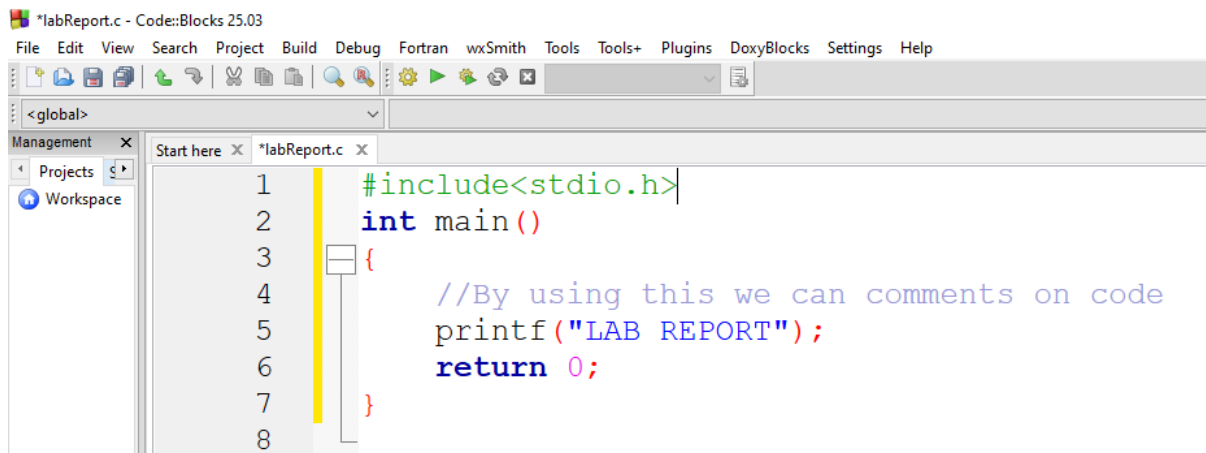
Here we will use CODEBLOCKS for writing our desired C program in our computer.

Open Codeblocks and go to *File > New > Empty File*.

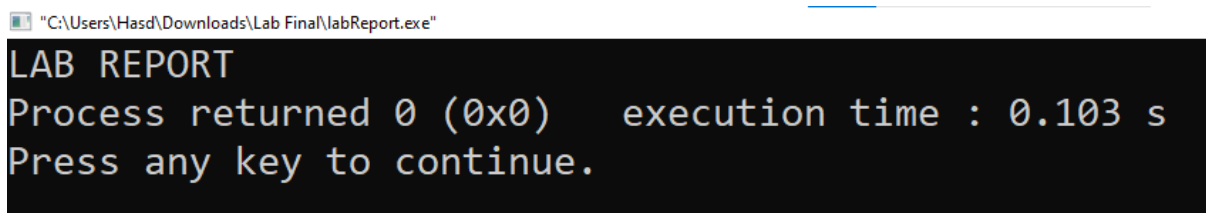
Write the following C code and save the file as **labReport.c** (*File > Save File as*):

Writing First Program in C Language

This simple program demonstrates the basic structure of a C program. It will also help us understand the basic syntax of a C program.

A screenshot of a code editor window titled '*labReport.c - Code::Blocks 25.03'. The menu bar includes File, Edit, View, Search, Project, Build, Debug, Fortran, wxSmith, Tools, Tools+, Plugins, DoxyBlocks, Settings, and Help. The toolbar contains icons for file operations, editing, and execution. The left sidebar shows 'Management' with 'Projects' and 'Workspace' sections. The main editor area shows a C program with line numbers 1 through 8. The code is: 1 #include<stdio.h>, 2 int main(), 3 {, 4 //By using this we can comments on code, 5 printf("LAB REPORT");, 6 return 0; , 7 }, 8. A yellow vertical line is at the end of line 1.

Then, go to **Build > Build and Run** to run (*execute*) the program. The result will look something to this:

A screenshot of a terminal window with the title bar '"C:\Users\Hasd\Downloads\Lab Final\labReport.exe"'. The terminal output is: LAB REPORT, Process returned 0 (0x0) execution time : 0.103 s, Press any key to continue.

Structure of the C program

After the above discussion, we can formally assess the basic structure of a C program. By structure, it is meant that any program can be written in this structure only. Writing a C program in any other structure will lead to a **Compilation Error**. The structure of a C program is as follows:

		1	#include<stdio.h>	Header
		2	int main()	Main
BODY		3	{	
		4	//by using this we can comment on code	Comment
		5	printf("LAB REPORT");	Statement
		6	return 0;	Return
		7	}	

Header Files Inclusion - Line 1 [#include <stdio.h>]

The first component is the Header files in a C program. A header file is a file with extension **.h** which contains C function declarations and macro definitions to be shared between several

source files. All lines that start with **#** are processed by a preprocessor which is a program invoked by the compiler. In the above example, the preprocessor copies the preprocessed code of **stdio.h** to our file. The .h files are called header files in C.

Some of the C Header files:

- ❖ **stddef.h** - Defines several useful types and macros.
- ❖ **stdint.h** - Defines exact width integer types.
- ❖ **stdio.h** - Defines core input and output functions
- ❖ **stdlib.h** - Defines numeric conversion functions, pseudo-random number generator, and memory allocation
- ❖ **string.h** - Defines string handling functions
- ❖ **math.h** - Defines common mathematical functions.

Main Method Declaration - Line 2 [int main()]

The next part of a C program is the main() function. It is the entry point of a C program and the execution typically begins with the first line of the main(). The empty brackets indicate that the main doesn't take any parameter (. The int that was written before the main indicates the return type of main(). The value returned by the main indicates the status of program termination.

Body of Main Method - Line 3 to Line 6 [enclosed in {}]

The body of the main method in the C program refers to statements that are a part of the main function. It can be anything like manipulations, searching, sorting, printing, etc. A pair of curly brackets define the body of a function. All functions must start and end with curly brackets.

Comment - Line 4[// This prints "By using this we can comments on code"]

The comments are used for the documentation of the code or to add notes in your program that are ignored by the compiler and are not the part of executable program .

Statement - Line 4 [printf("LAB REPORT");]

Statements are the instructions given to the compiler. In C, a statement is always terminated by a semicolon (;). In this particular case, we use printf() function to instruct the compiler to display "LAB REPORT" text on the screen.

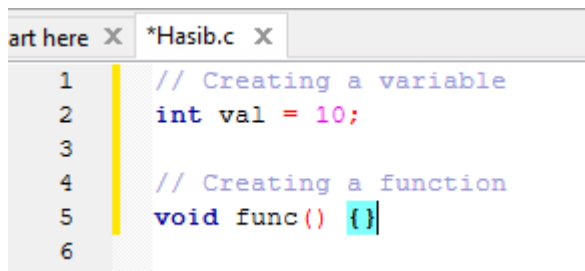
Return Statement - Line 5 [return 0;]

The last part of any C function is the return statement. The return statement refers to the return values from a function. This return statement and return value depend upon the return type of the function. The return statement in our program returns the value from main(). The returned value may be used by an operating system to know the termination status of your program. The value 0 typically means successful termination.

Identifiers in C

The names used to identify variables, functions, arrays, structures, and other user-defined objects in C programming are known as identifiers. A program element is uniquely identified by its name, which can be used to refer to it at a later stage of the program.

Example:

A screenshot of a code editor window titled '*Hasib.c'. The editor shows a C program with the following code:

```
1 // Creating a variable
2 int val = 10;
3
4 // Creating a function
5 void func() {}
6
```

The code is color-coded: comments are in blue, keywords like 'int' and 'void' are in purple, and the variable 'val' and function 'func' are in black. The cursor is at the end of line 5.

"val" and "func" are identifiers in the code sample above.

Rules for Naming Identifiers in C

A programmer must follow a set of rules to create an identifier in C:

- Identifier can contain following characters:
 - Uppercase (A-Z) and lowercase (a-z) alphabets.
 - Numeric digits (0-9).
 - Underscore (_).
- The first character of an identifier must be a letter or an underscore.
- Identifiers are case-sensitive.
- Identifiers cannot be keywords in C (such as int, return, if, while etc.).

Naming Conventions

In C programming, naming conventions are not strict rules but are commonly followed suggestions by the programming community for identifiers to improve readability and understanding of code. Below are some conventions that are commonly used:

For Variables:

- Use camelCase for variable names (e.g., frequencyCount, personName).
- Constants can use UPPER_SNAKE_CASE (e.g., MAX_SIZE, PI).
- Start variable names with a lowercase letter.
- Use descriptive and meaningful names.

For Functions:

- Use camelCase for function names (e.g., getName(), countFrequency()).
- Function names should generally be verbs or verb phrases that describe the action.

C VARIABLE

A variable in C is a named piece of memory which is used to store data and access it whenever required.

- It allows us to use the memory without having to memorize the exact memory address.
- To create a variable in C, we have to specify a name and the type of data it is going to store.
- C provides different data types that can store almost all kinds of data. For example, int, char, float, double, etc.
- Every variable must be declared before it is used. We can also declare multiple variables of same data type in a single statement by separating them using comma.

Data Type	Size	Description	Example
<i>int</i>	2 or 4 bytes	Stores whole numbers, without decimals	1
<i>float</i>	4 bytes	Stores fractional numbers, containing one or more decimals. Sufficient for storing 6-7 decimal digits	1.99
<i>double</i>	8 bytes	Stores fractional numbers, containing one or more decimals. Sufficient for storing 15 decimal digits	1.99
<i>char</i>	1 bytes	Stores a single character/letter/number, or ASCII values	'A'

Rules for Naming Variables in C

- We can assign any name to a C variable as long as it follows the following rules:
- A variable name must only contain letters, digits, and underscores.
- It must start with an alphabet or an underscore only. It cannot start with a digit.
- No white space is allowed within the variable name.
- A variable name must not be any reserved word or keyword.
- The name must be unique in the program.

```
Start here X *Hasib.c X
1  #include <stdio.h>
2
3  int main()
4  {
5      int age = 20; // integer variable
6      float height = 5.7; // floating-point variable
7      char grade = 'A'; // character variable
8
9      printf("Age: %d\n", age); // %d Format Specifiers for char
10     printf("Height: %.1f\n", height); // %f Format Specifiers for float
11     printf("Grade: %c\n", grade); // %c Format Specifiers for char
12
13     return 0;
14 }
```

OUTPUT:

```
Age: 20
Height: 5.7
Grade: A

Process returned 0 (0x0)   execution time : 0.052 s
Press any key to continue.
```

Basic Input and Output in C

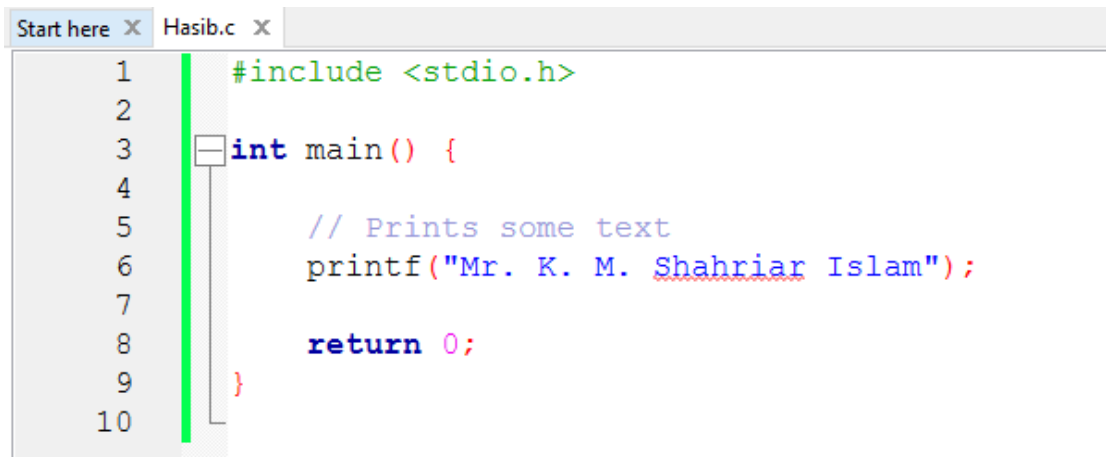
In C, there are many input and output for different situations, but the most commonly used functions for Input/Output are *scanf()* and *printf()* respectively. These functions are part of the standard input/output library `<stdio.h>`. *scanf()* takes user inputs (typed using keyboard) and *printf()* displays output on the console or screen.

Basic Output in C

The *printf()* function is used to print formatted output to the standard output `stdout` (which is generally the console screen). It is one of the most commonly used functions in C.

The following examples demonstrate the use of *printf* for output in different cases:

Printing Some Text:



```
Start here x Hasib.c x
1  #include <stdio.h>
2
3  int main() {
4
5      // Prints some text
6      printf("Mr. K. M. Shahriar Islam");
7
8      return 0;
9  }
10
```

OUTPUT:

```
Mr. K. M. Shahriar Islam
Process returned 0 (0x0)   execution time : 0.056 s
Press any key to continue.
```

Explanation: The text inside "" is called a string in C. It is used to represent textual information. We can directly pass strings to the *printf()* function to print them in console screen.

Basic Input in C:

scanf() is used to read user input from the console. It takes the format string and the addresses of the variables where the input will be stored.

Syntax

```
scanf("formatted_string", address_of_variables/values);
```

This function takes the address of the arguments where the read value is to be stored.

Examples of Reading User Input

```
Start here X Hasib.c X
1  #include <stdio.h>
2
3  int main() {
4      int student_id;
5      printf("Enter your student_id: ");
6
7      // Reads an integer
8      scanf("%d", &student_id);
9
10     // Prints the student id
11     printf("Student Id is: %d\n", student_id);
12     return 0;
13 }
14
```

OUTPUT:

```
"C:\Users\Hasd\Downloads\Lab Final\Hasib.exe"
Enter your student_id: 253
Student Id is: 253
```

Explanation: %d is used to read an integer; and &student_id provides the address of the variable where the input will be stored.

Operators in C

Operators are the basic components of C programming. They are symbols that represent some kind of operation, such as mathematical, relational, bitwise, conditional, or logical computations, which are to be performed on values or variables. The values and variables used with operators are called operands.

Unary, Binary and Ternary Operators

On the basis of the number of operands they work on, operators can be classified into three types:

1. Unary Operators: Operators that work on single operand.
Example: Increment(++), Decrement(--)
2. Binary Operators: Operators that work on two operands.
Example: Addition (+), Subtraction(-), Multiplication (*)
3. Ternary Operators: Operators that work on three operands.
Example: Conditional Operator(?:)

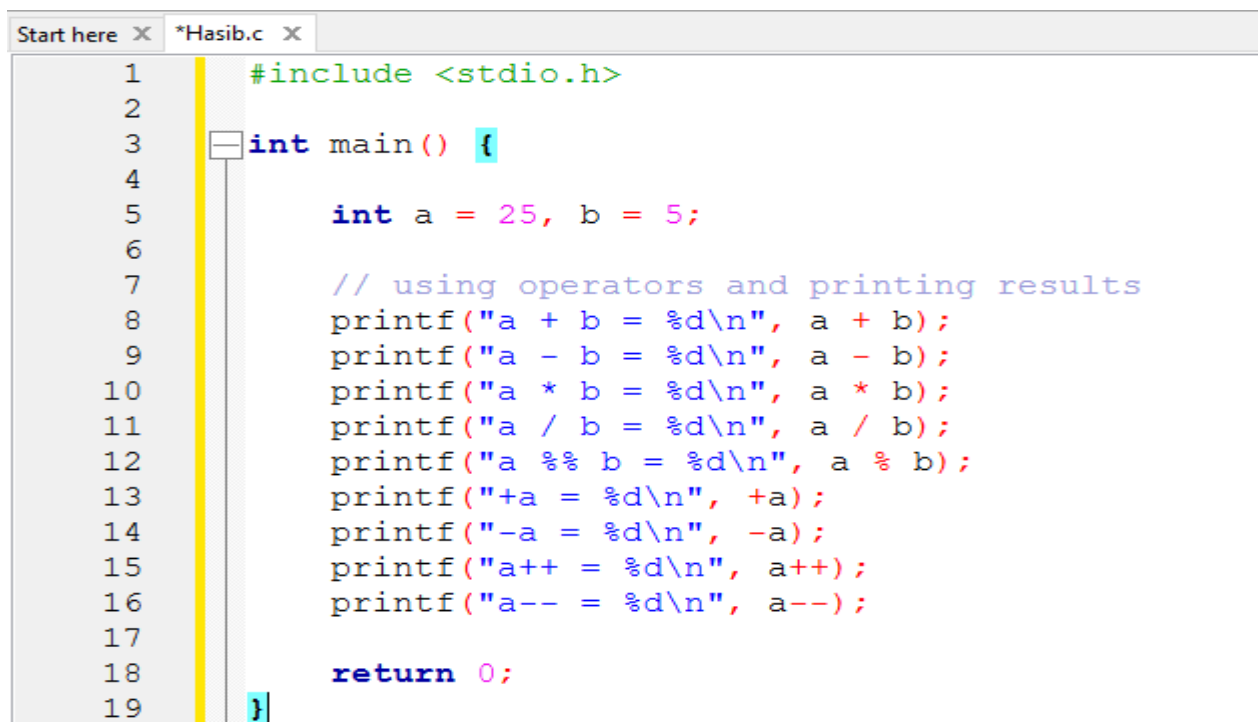
Types of Operators in C

C language provides a wide range of built in operators that can be classified into 6 types based on their functionality:

1. Arithmetic Operators
2. Relational Operators
3. Logical Operator
4. Bitwise Operators
5. Assignment Operators
6. Other Operators (sizeof Operator, conditional operator)

Arithmetic Operators

The arithmetic operators are used to perform arithmetic/mathematical operations on operands. There are 9 arithmetic operators in C language:



The screenshot shows a C program in a code editor with the following code:

```
1  #include <stdio.h>
2
3  int main() {
4
5      int a = 25, b = 5;
6
7      // using operators and printing results
8      printf("a + b = %d\n", a + b);
9      printf("a - b = %d\n", a - b);
10     printf("a * b = %d\n", a * b);
11     printf("a / b = %d\n", a / b);
12     printf("a %% b = %d\n", a % b);
13     printf("+a = %d\n", +a);
14     printf("-a = %d\n", -a);
15     printf("a++ = %d\n", a++);
16     printf("a-- = %d\n", a--);
17
18     return 0;
19 }
```


OUTPUT:

```
"C:\Users\Hasd\Downloads\Lab Final\Hasib.exe"
a + b = 30
a - b = 20
a * b = 125
a / b = 5
a % b = 0
+a = 25
-a = -25
a++ = 25
a-- = 26
```

Symbol	Operator	DESCRIPTON	SYNTAX
&&	Logical AND	Returns true if both the operands are true.	a && b
	Logical OR	Returns true if both or any of the operand is true.	a b
!	Logical NOT	Returns true if the operand is false.	a b
=	Simple Assignment	Assign the value of the right operand to the left operand	a = b
+=	Plus and assign	Add the right operand and left operand and assign this value to the left operand.	a += b

Decision Making in C

In C, programs can choose which part of the code to execute based on some condition. This ability is called decision making and the statements used for it are called conditional statements. These statements evaluate one or more conditions and make the decision whether to execute a block of code or not.

1. if in C

The if statement is the simplest decision-making statement. It is used to decide whether a certain statement or block of statements will be executed or not i.e if a certain condition is true then a block of statements is executed otherwise not.

A condition is any expression that evaluates to either a true or false (or values convertible to true or flase).

```
Start here x *Hasib.c x
1      #include <stdio.h>
2
3      int main() {
4
5          int age = 20;
6
7          // If statement
8          if (age >= 18) {
9              printf("Eligible for vote");
10         }
11     }
12
```

OUTPUT:

```
"C:\Users\Hasd\Downloads\Lab Final\Hasib.exe"
Eligible for vote
Process returned 0 (0x0)   execution time : 0.050 s
Press any key to continue.
```

The expression inside () parenthesis is the condition and set of statements inside {} braces is its body. If the condition is true, only then the body will be executed.

2. if-else in C

The if statement alone tells us that if a condition is true, it will execute a block of statements and if the condition is false, it won't. But what if we want to do something else when the condition is false? Here comes the C else statement. We can use the else statement with the if statement to execute a block of code when the condition is false. The if-else statement consists of two blocks, one for false expression and one for true expression.

```
Start here x Hasib.c x
1      #include <stdio.h>
2
3      int main() {
4          int age = 10;
5
6          if (age >= 18) {
7              printf("Eligible for vote");
8          }
9          else {
10             printf("Not Eligible for vote");
11         }
12         return 0;
13     }
14
```

OUTPUT:

```
"C:\Users\Hasd\Downloads\Lab Final\Hasib.exe"
Not Eligible for vote
Process returned 0 (0x0)   execution time : 0.050 s
Press any key to continue.
```

The block of code following the else statement is executed as the condition present in the if statement is false.

3.Nested if-else in C

A nested if in C is an if statement that is the target of another if statement. Nested if statements mean an if statement inside another if statement. Yes, C allow us to nested if statements within if statements, i.e, we can place an if statement inside another if statement.

```
here X *Hasib.c X
1  #include <stdio.h>
2
3  int main(){
4      int age = 11;
5
6      if (age >= 18) {
7          if (age >= 60)
8              printf("Eligible to vote (Senior Citizen)\n");
9          else
10             printf("Eligible for vote\n");
11     }
12     else {
13         printf("Not eligible to vote (Under 18)\n");
14         if (age >= 13)
15             printf("teenager\n");
16         else
17             printf("not a teenager\n");
18     }
19
20     return 0;
21 }
22
```

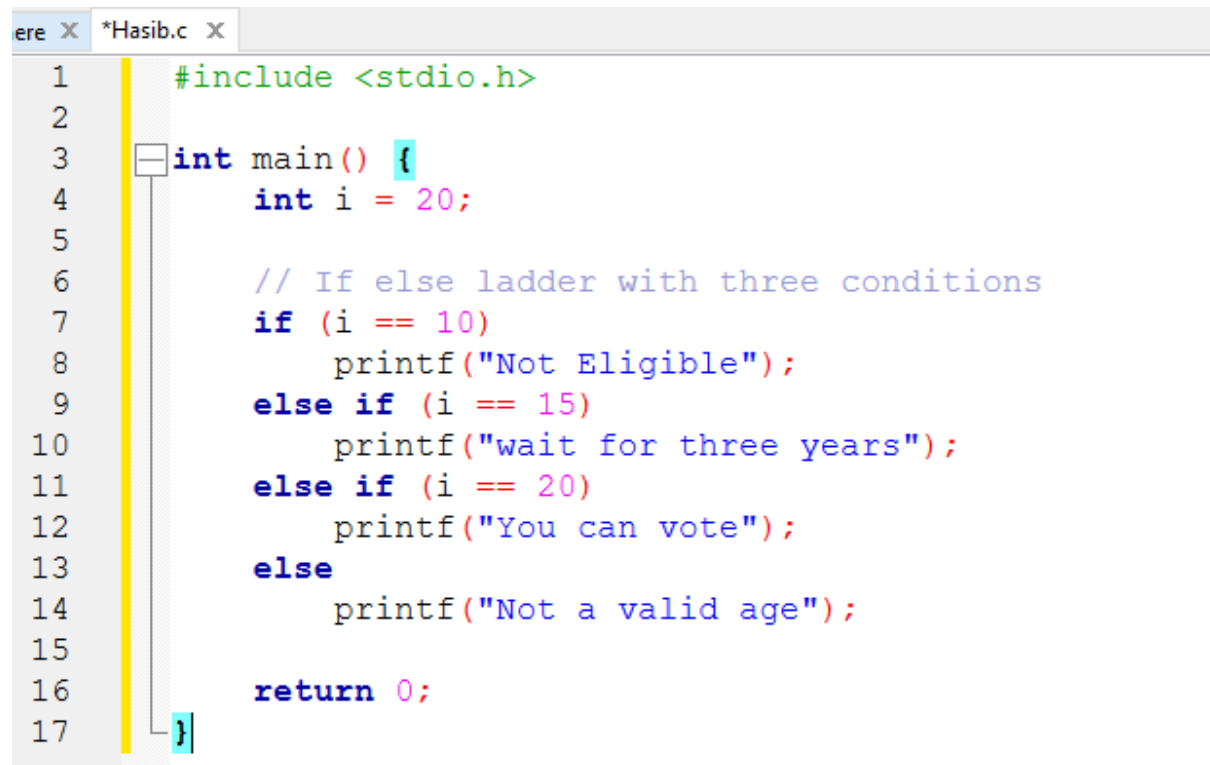
OUTPUT:

```
"C:\Users\Hasd\Downloads\Lab Final\Hasib.exe"
Not eligible to vote (Under 18)
not a teenager

Process returned 0 (0x0)   execution time : 0.046 s
Press any key to continue.
```

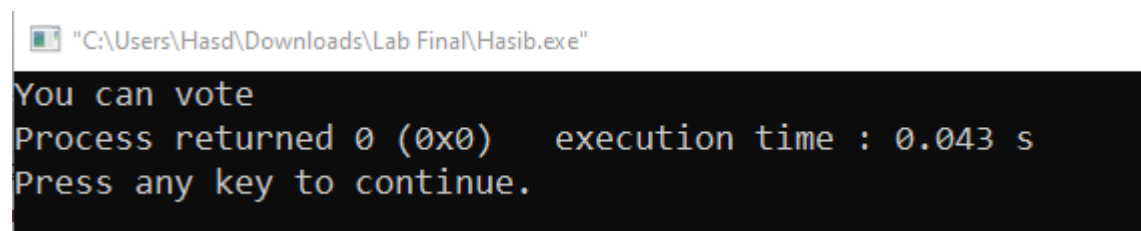
4. if-else-if Ladder in C

The if else if statements are used when the user has to decide among multiple options. The C if statements are executed from the top down. As soon as one of the conditions controlling the if is true, the statement associated with that if is executed, and the rest of the C else-if ladder is bypassed. If none of the conditions is true, then the final else statement will be executed. if-else-if ladder is similar to the switch statement.

A screenshot of a code editor window titled '*Hasib.c'. The code is as follows:

```
1  #include <stdio.h>
2
3  int main() {
4      int i = 20;
5
6      // If else ladder with three conditions
7      if (i == 10)
8          printf("Not Eligible");
9      else if (i == 15)
10         printf("wait for three years");
11     else if (i == 20)
12         printf("You can vote");
13     else
14         printf("Not a valid age");
15
16     return 0;
17 }
```

OUTPUT:

A screenshot of a command prompt window showing the execution of the program. The output is as follows:

```
"C:\Users\Hasd\Downloads\Lab Final\Hasib.exe"
You can vote
Process returned 0 (0x0)   execution time : 0.043 s
Press any key to continue.
```

Ultimately, both if-else and nested if-else statements are essential C language tools for directing a program's flow by selecting which code to run based on runtime conditions, with nesting providing the ability to manage increasingly intricate, multi-layered logical decisions.

Swap Two Numbers Without Using Third Variable:

```
art here x Hasib.c x
1      #include <stdio.h>
2
3      int main() {
4          int a,b;
5          scanf("%d %d", &a, &b);
6
7          //Before swap
8          printf("a=%d , b=%d\n",a,b);
9
10         a=(a+b)-(b=a);
11
12         //After swap
13         printf("a=%d , b=%d",a,b);
14
15         return 0;
16     }
17
```

OUTPUT:

```
"C:\Users\Hasd\Downloads\Lab Final\Hasib.exe"
10 20
a=10 , b=20
a=20 , b=10
Process returned 0 (0x0)   execution time : 2.256 s
Press any key to continue.
```